

Welcome to 151B: Deep Learning

Instructor: Annapurna Vadaparty

TA: Ben Bao

Tutors: Alexander Radulescu, Trevor Duong

Many slides from Prof. Gary Cottrell “Recurrent Networks Part 1”, which includes slides from Prof. Geoff Hinton “Recurrent neural networks”

Grading Questions

- To get a HP on an assignment you need a HP (on all parts of the assignment):
 - Programming: 1) your submission meeting benchmarks, and 2) both questions of your oral assessment
 - Written portion
 - E.g. for a HP for A1, you'd be expected to get a HP on the written portions (90% overall for the written parts of A1 parts 1 and 2), and also a HP on the programming portions (both meeting the benchmarks as specified, and both questions on the oral assessment).
- Exams: Unfortunately we won't be able to release them as they may be in use for a concurrently-running course for a different organization. If there are specific topics questions you'd like to ask about, I'd be happy to chat during office hours!
 - Lots of hidden decisions/work intended to make your experience smoother (e.g. textbook, GPUs over the summer)--sometimes it doesn't work out! But you're always welcome to ask. I'm on your team!

Great job on following autograder instructions for A1!

Keep it up for A2! Makes grading ~~easier~~ possible.

For receiving your A1 Part 1 grade–fill out the T/F question on PrairieLearn under (this lets me input a grade).

Enjoying your LPRRs!

Glad to hear that people are trying to make sure things make sense.

Seeing some “don’t have it create code for math you don’t understand” – great!

Other sources (e.g. Wikipedia, Andrej Karpathy’s blog)–underrated!

Representing Time

- Time is obviously important
- How do we represent time in neural networks?
- Two approaches:
 - Map time into **space**
 - Map time into the **state** of the network

Notation: Inputs and Time Steps

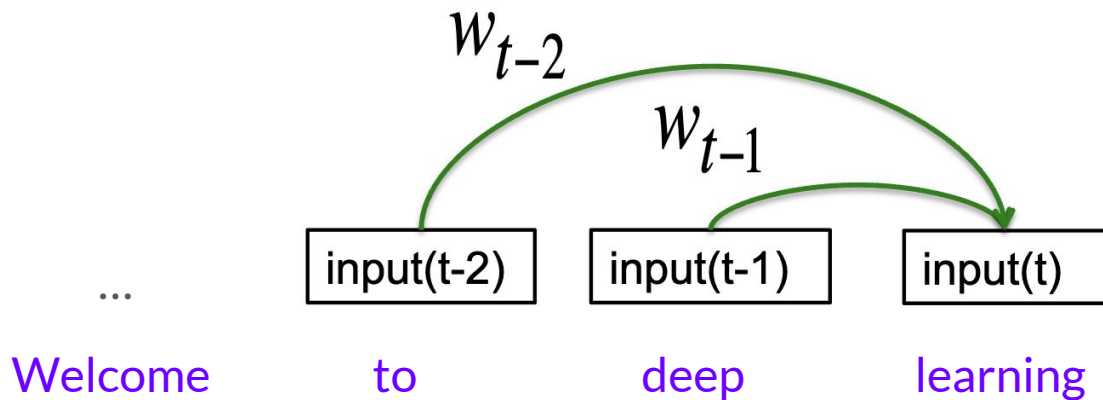
“Welcome to deep learning” mapped to input locations:

Welcome	to	deep	learning
t_1	t_2	t_3	t_4

Mapping Time into **Space**:

Input Locations Correspond to Time Step

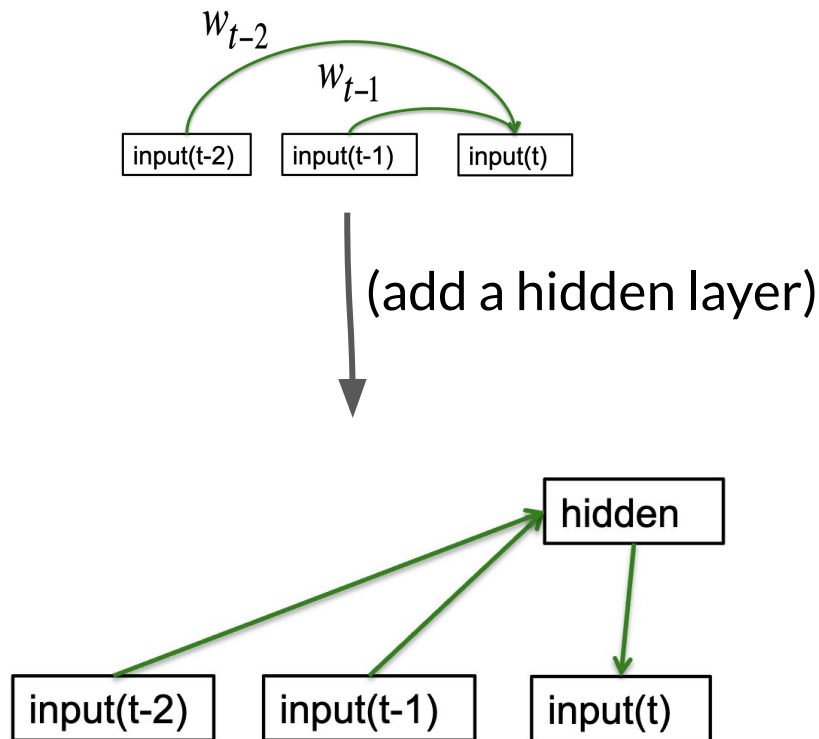
- You could predict the next term in a sequence from a fixed number of previous terms:



- This is called an “autoregressive model”, and it is a commonly used technique.
- Limitations?** Hint: If each of the words is an input node, how many input nodes?

You could even try this with an MLP...

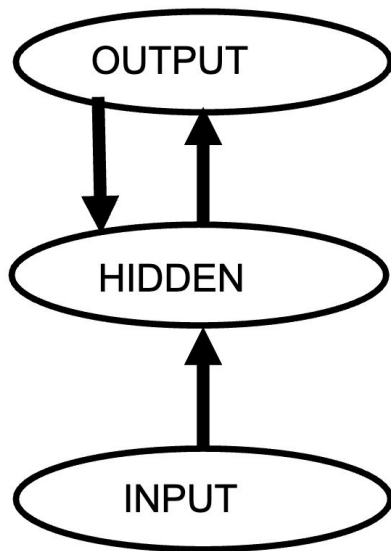
- ...but you'd run into the same issue: fixed number of inputs
- Works for simple problems
- Doesn't work so well for arbitrary length items - e.g., sentences.



Mapping Time Into **State**: Explicitly Storing Information About Inputs

Jordan Networks

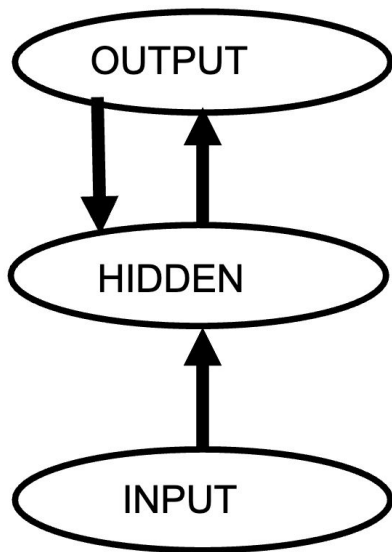
(A very simple recurrent network–will help us build up to RNNs/LSTMs)



Hidden units are updated with information from the **output** of the previous timestep (keeps track of what it just did)

MLPs also have “hidden units”. What’s the difference?

Jordan Networks

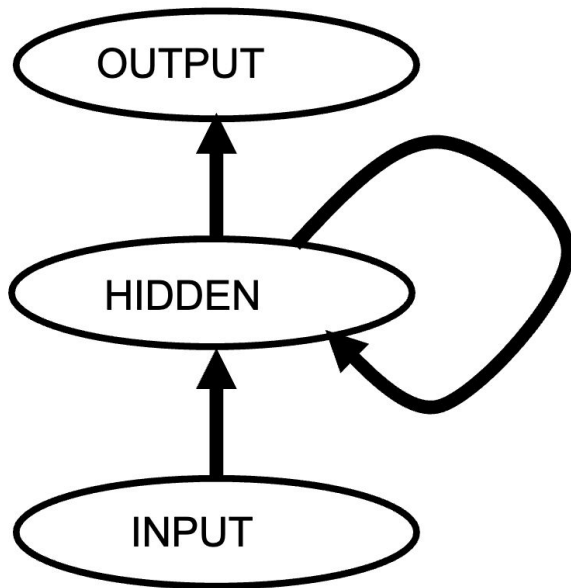


- All of the arrows in this picture are part of just the processing of an input, not backprop—storing information **about each step** is part of the process
- If you look at a hidden state in a Jordan network, you get some information about the last output created. This is not true of an MLP—there, the weights represent aggregate info learned about all the input you’ve seen so far.
- Imagine you had a sticky note about the last item you generated in a sequence

Mapping Time Into State: Explicitly Storing Information About Inputs

Elman Networks

(simple recurrent network–SRN)

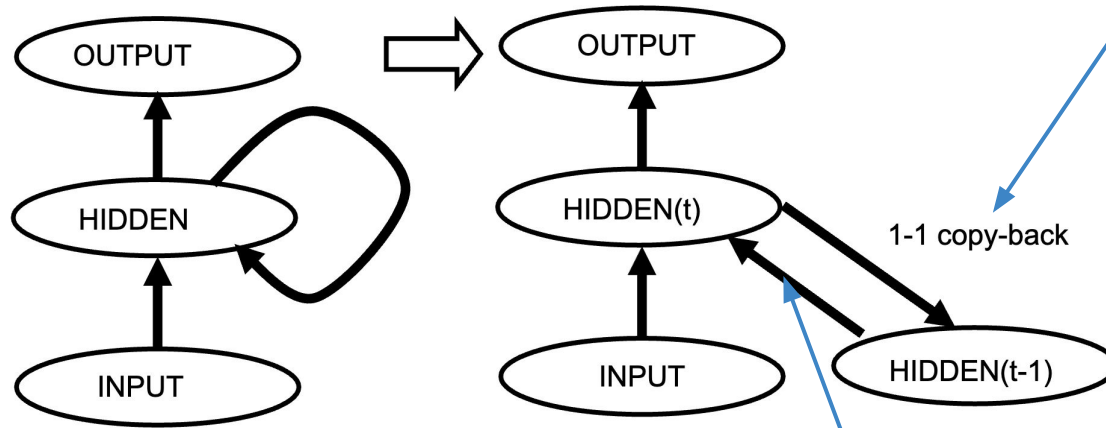


Hidden units are updated with information from that same **hidden unit's** information of the previous timestep

Allows it to learn from more than just the past outputs–flexible in what it retains

Questions so far? Next we'll talk about how these learn...

Let's unpack the recurrent arrow.

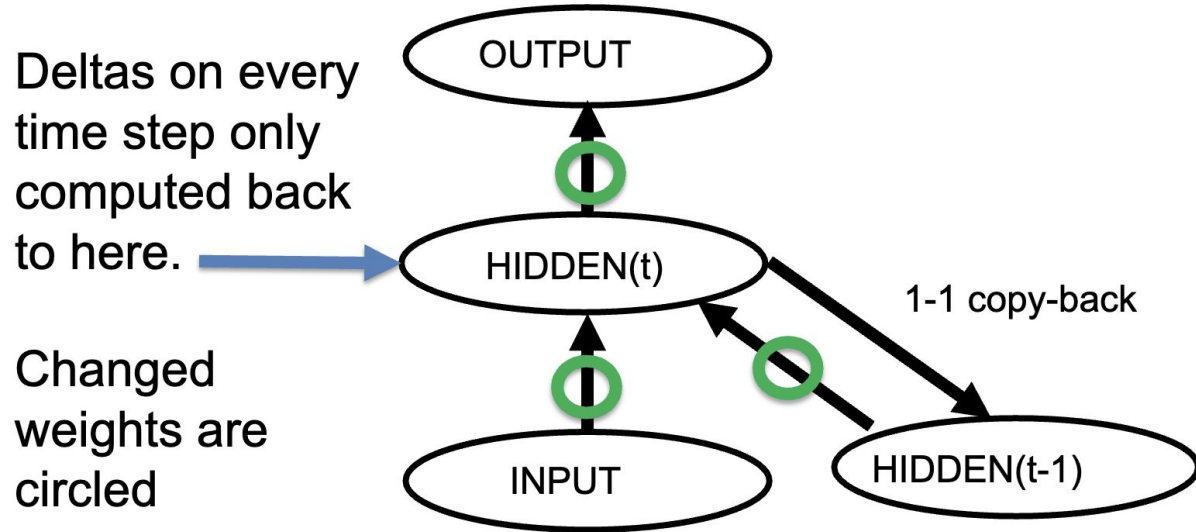


Makes an identical copy of the hidden state at time t

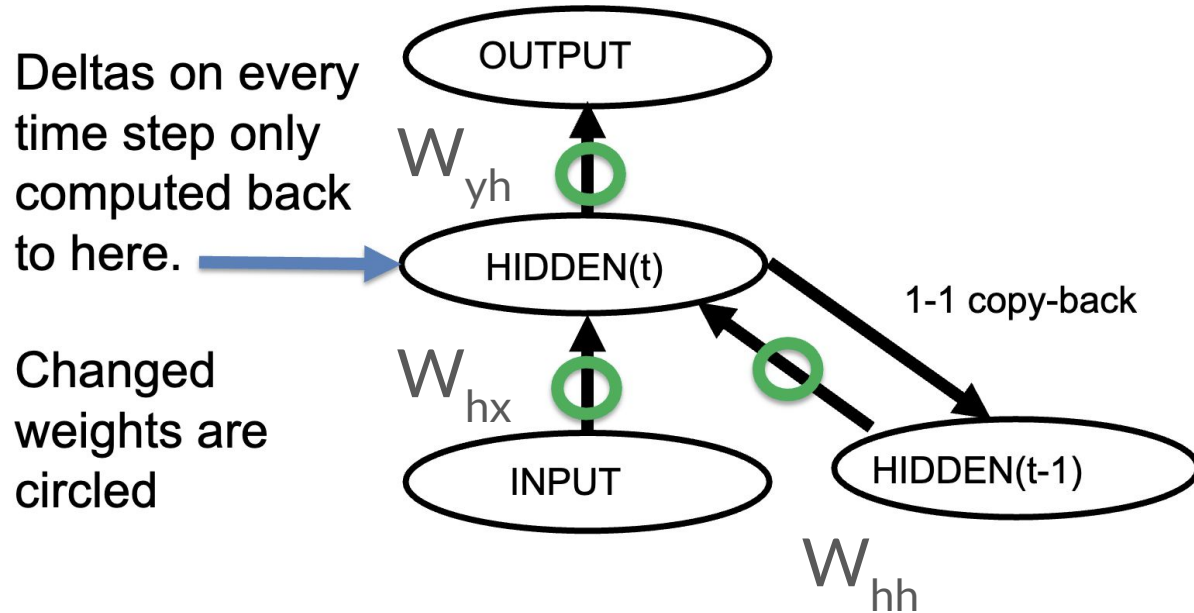
Something interesting (W_{hh})

Now, let's do backprop on it—but just for one timestep

- The original implementation of these SRNs only did one step BPTT



Matrices



Finding Structure in Time (Elman, 1990)

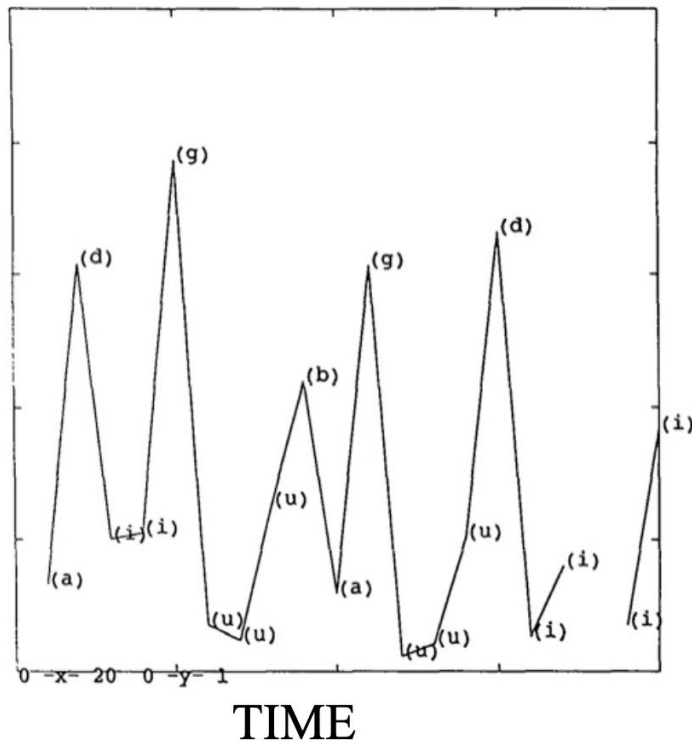
Let's start learning from papers—this will be an increasingly useful skill set as you try to keep up with newer models.

- Elman's network was used for predicting just one next element (character)
- Start with “words”: **ba, dii, guuu**
- Randomly generate a sequence of these words and ask the network to predict the next letter.
 - E.g.: ba guuu dii dii ba guuu dii
- **Which next letters do you think it will do well on? And poorly?**

Finding Structure in Time (Elman, 1990)

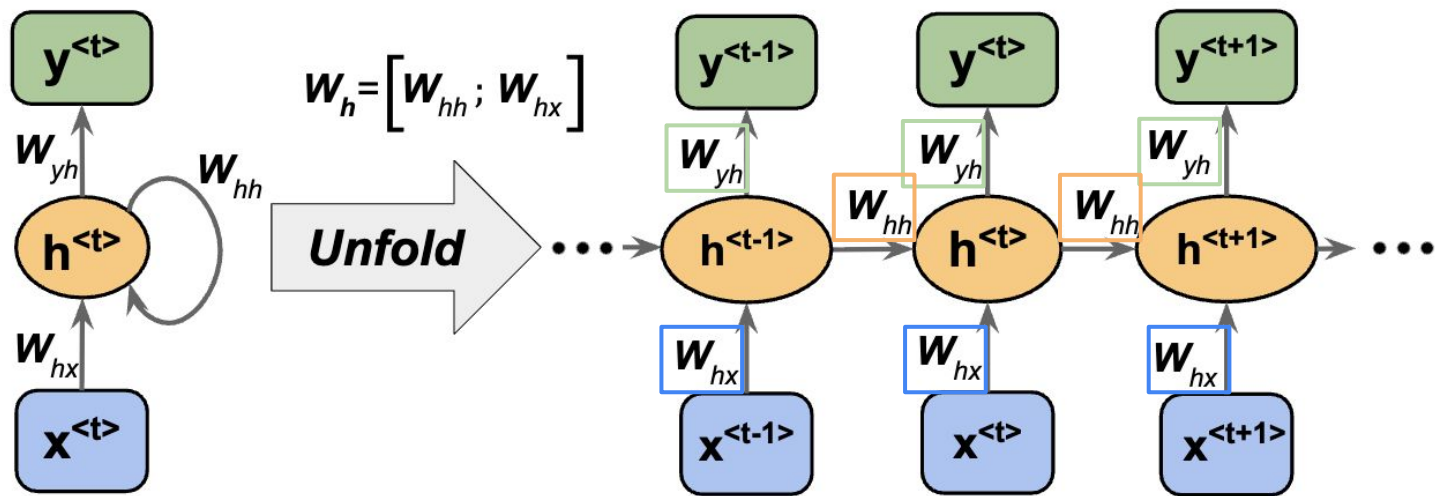
Results:

Output Error



- Words: **ba, dii, guuu**
- Sequences such as: **ba guuu dii dii ba guuu dii**
- What do you notice about the steps it does well at? And poorly?
- Learned words without being told where to break!

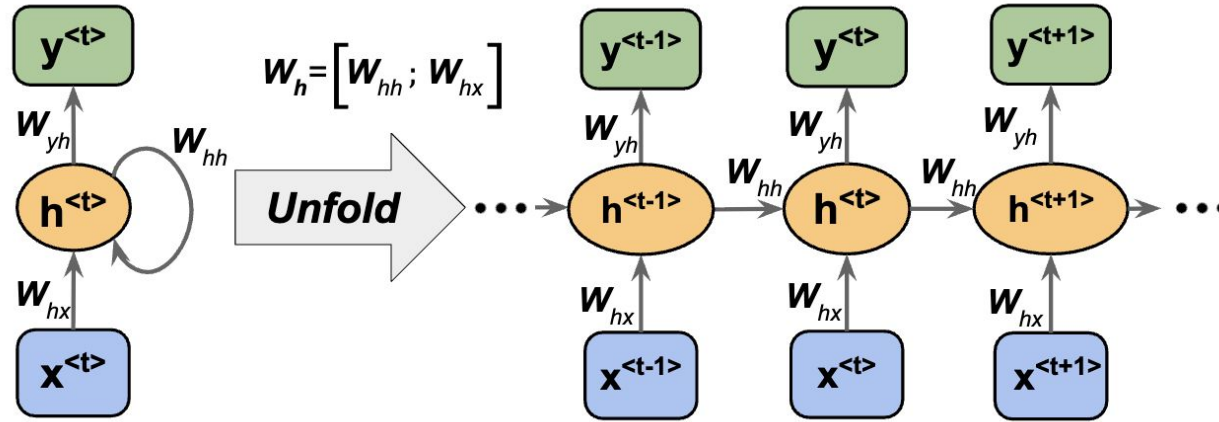
RNNs Feedforward Pass (Using >1 Time Steps)



Weight matrices are the same across each timestep

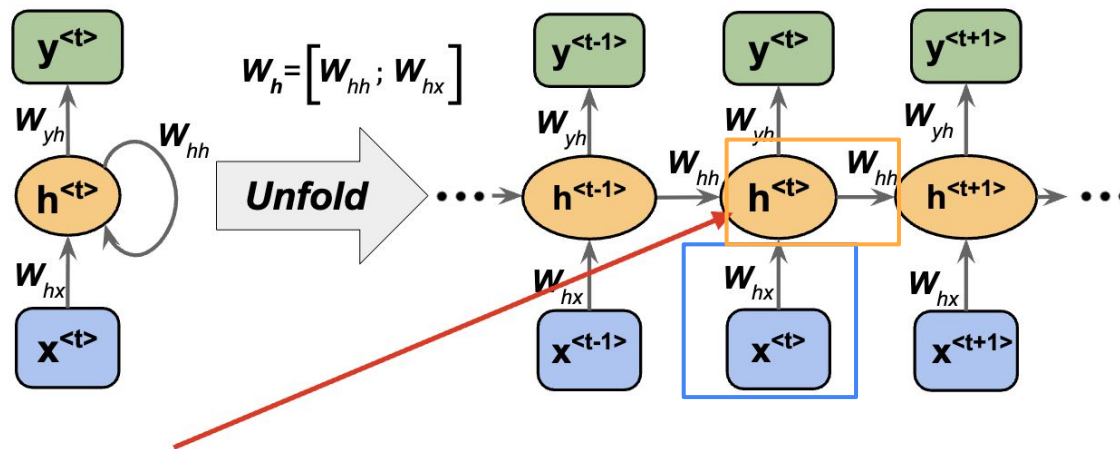
RNN Forward Pass

“A recurrent network is just a special case of a feedforward network”– Dave Rumelhart



More precisely: For a fixed T , an unrolled RNN is a depth- T feedforward network with weights tied across layers and an new input injected at each layer. So, training it is ordinary backprop, with shared-weight gradients summed across timesteps.

RNN Forward Pass: Input to Hidden



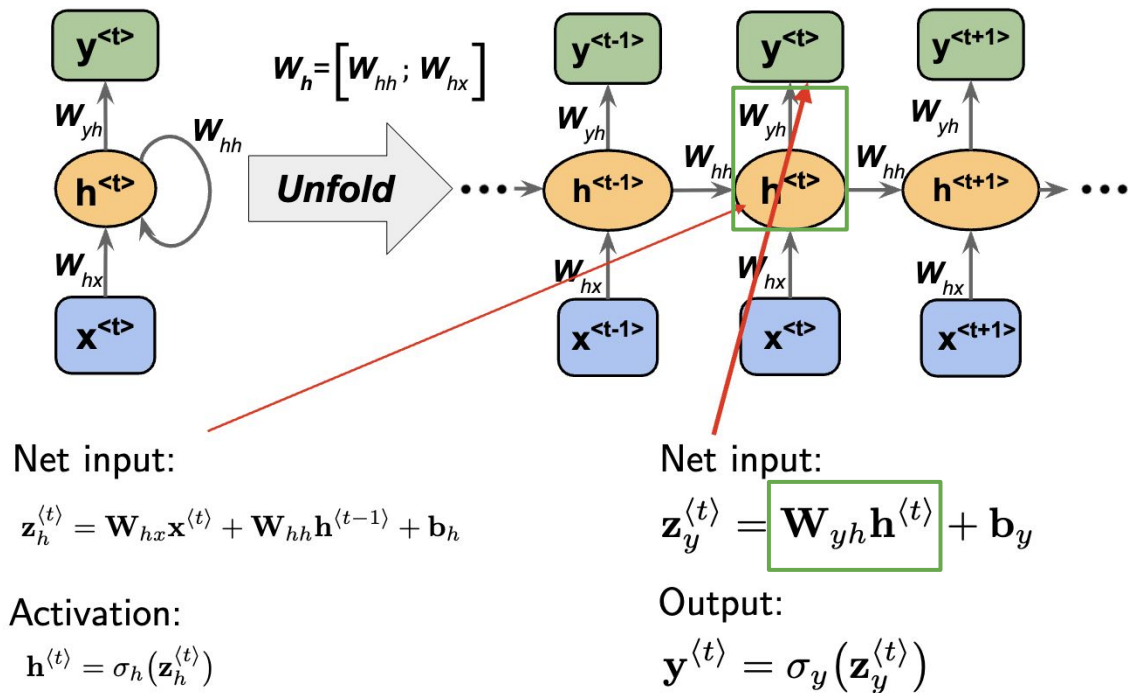
Net input:

$$\mathbf{z}_h^{(t)} = \mathbf{W}_{hx} \mathbf{x}^{(t)} + \mathbf{W}_{hh} \mathbf{h}^{(t-1)} + \mathbf{b}_h$$

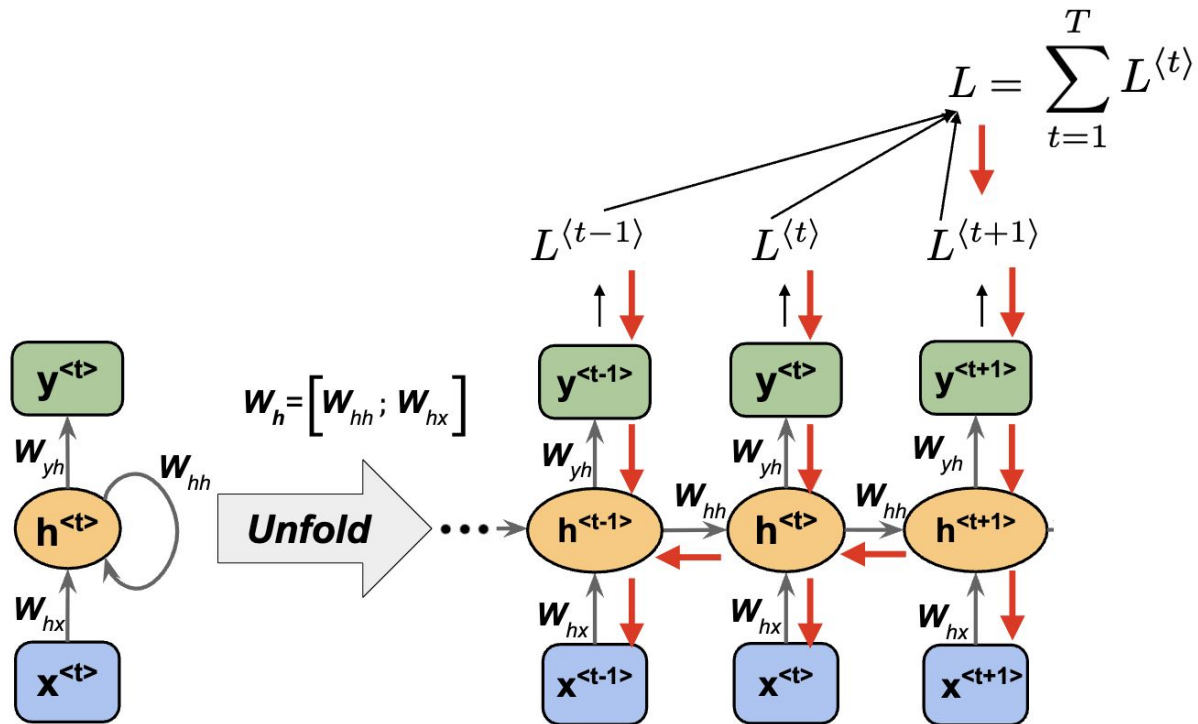
Activation:

$$\mathbf{h}^{(t)} = \sigma_h(\mathbf{z}_h^{(t)})$$

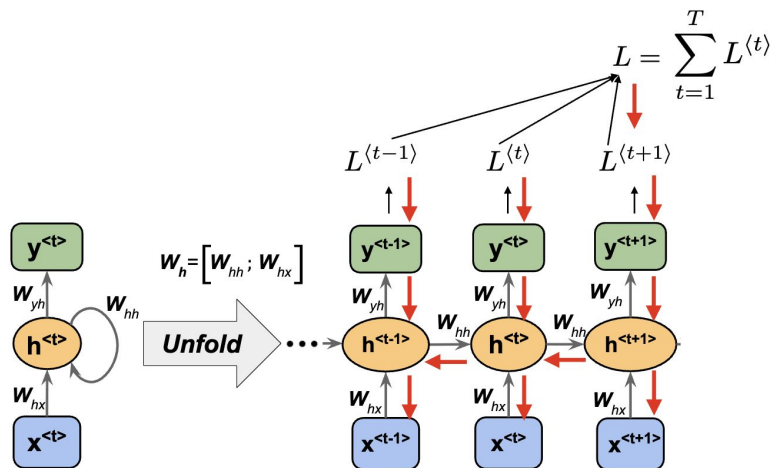
RNN Forward Pass: Hidden to Output



Backpropagation through time



Backpropagation through time



We also compute $\frac{\partial L^{(t)}}{\partial W_{yh}}$ and $\frac{\partial L^{(t)}}{\partial W_{hx}}$,
but we'll take a close look at $\frac{\partial L^{(t)}}{\partial W_{hh}}$.

$$L = \sum_{t=1}^T L^{(t)}$$

$$\frac{\partial L^{(t)}}{\partial W_{hh}} = \frac{\partial L^{(t)}}{\partial y^{(t)}} \cdot \frac{\partial y^{(t)}}{\partial h^{(t)}} \cdot \left(\sum_{k=1}^t \frac{\partial h^{(t)}}{\partial h^{(k)}} \cdot \frac{\partial h^{(k)}}{\partial W_{hh}} \right)$$

For every timestep...we're multiplying by gradients from *all previous timesteps*

BPTT: Vanishing/Exploding Gradient

$$L = \sum_{t=1}^T L^{(t)} \quad \frac{\partial L^{(t)}}{\partial \mathbf{W}_{hh}} = \frac{\partial L^{(t)}}{\partial y^{(t)}} \cdot \frac{\partial y^{(t)}}{\partial \mathbf{h}^{(t)}} \cdot \left(\sum_{k=1}^t \boxed{\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}}} \cdot \frac{\partial \mathbf{h}^{(k)}}{\partial \mathbf{W}_{hh}} \right)$$

computed as a multiplication of adjacent time steps:

This is very problematic:

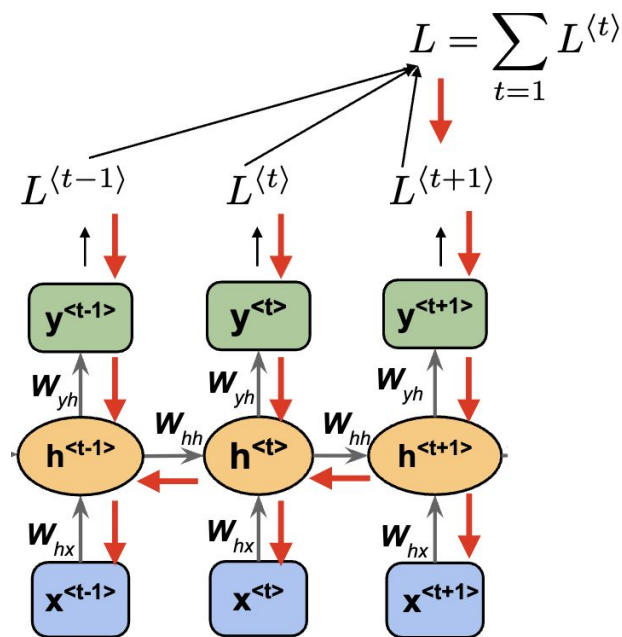
Vanishing/Exploding gradient problem!

$$\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(k)}} = \prod_{i=k+1}^t \frac{\partial \mathbf{h}^{(i)}}{\partial \mathbf{h}^{(i-1)}}$$

Another phrasing: # Timesteps = # Layers in an MLP

Computationally, this is like having an extra layer in an MLP for every input timestep you have.

Number of layers quickly becomes large—lots of extra weights.



Possible Solutions to Vanishing/Exploding gradient

- Gradient Clipping: set a max value for gradients
 - Only solves exploding gradient problem, not vanishing gradient
- Truncation: limits the number of time steps in backpropagation. For example, in a long sequence, only backpropagate through ~20-30
 - Misses out on longer-range dependencies

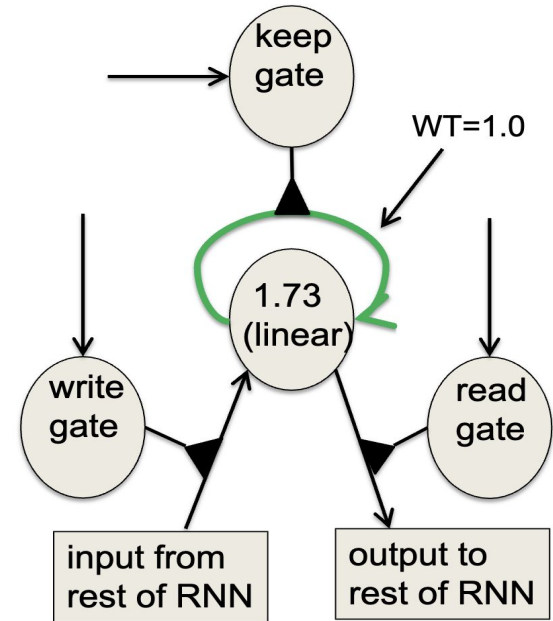
Instead, let's try:

- Long short-term memory (LSTM) -- uses a memory cell that can hold onto information about long-range dependencies and avoids vanishing gradient problems

Turning an RNN into an LSTM: Only Keep Some Information

To preserve information for a long time in the activities of an RNN, we use a circuit that implements an analog memory cell.

- A linear unit that has a self-link with a weight of 1 will maintain its state.
- Information is stored in the cell by activating its write gate.
- Information is retrieved by activating the read gate.



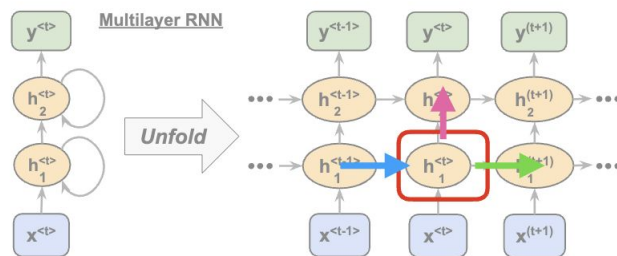
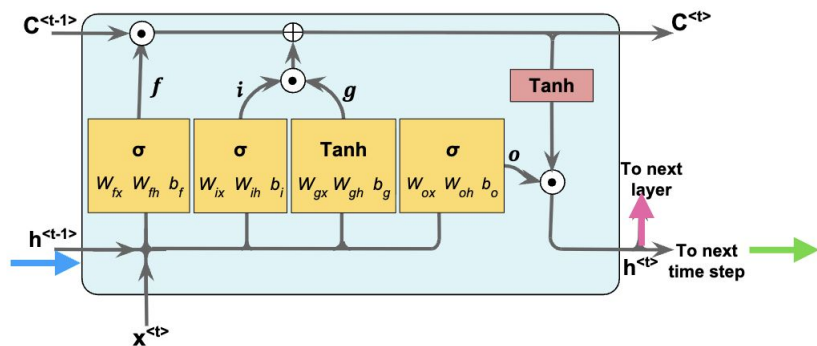
Turning an RNN into an LSTM: Only Keep Some Information

Some places we can “gate”/not use certain information:

- New input: Decide how much of the new input gets stored
 - Input modulation gate
- Old inputs: Decide how much to get rid of (forget)
 - Forget gate

Each of these will be a function (sigmoid, tanh) that takes weight matrices...which means you can change (learn!) how much you take in/keep at each step

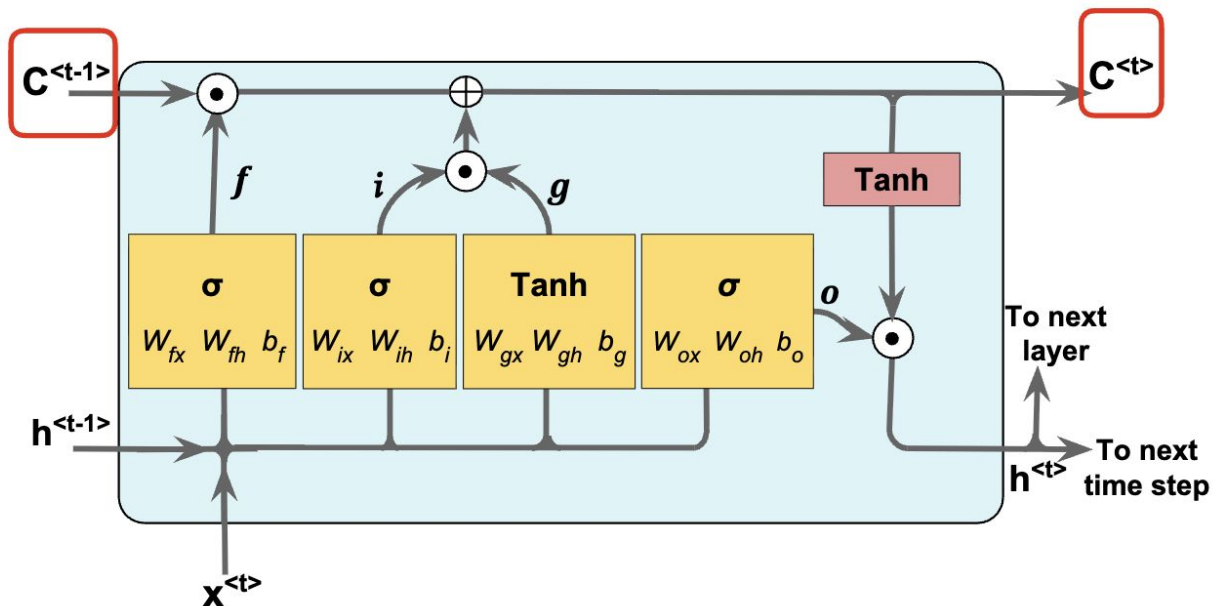
An LSTM Cell (replaces a hidden layer in an RNN)



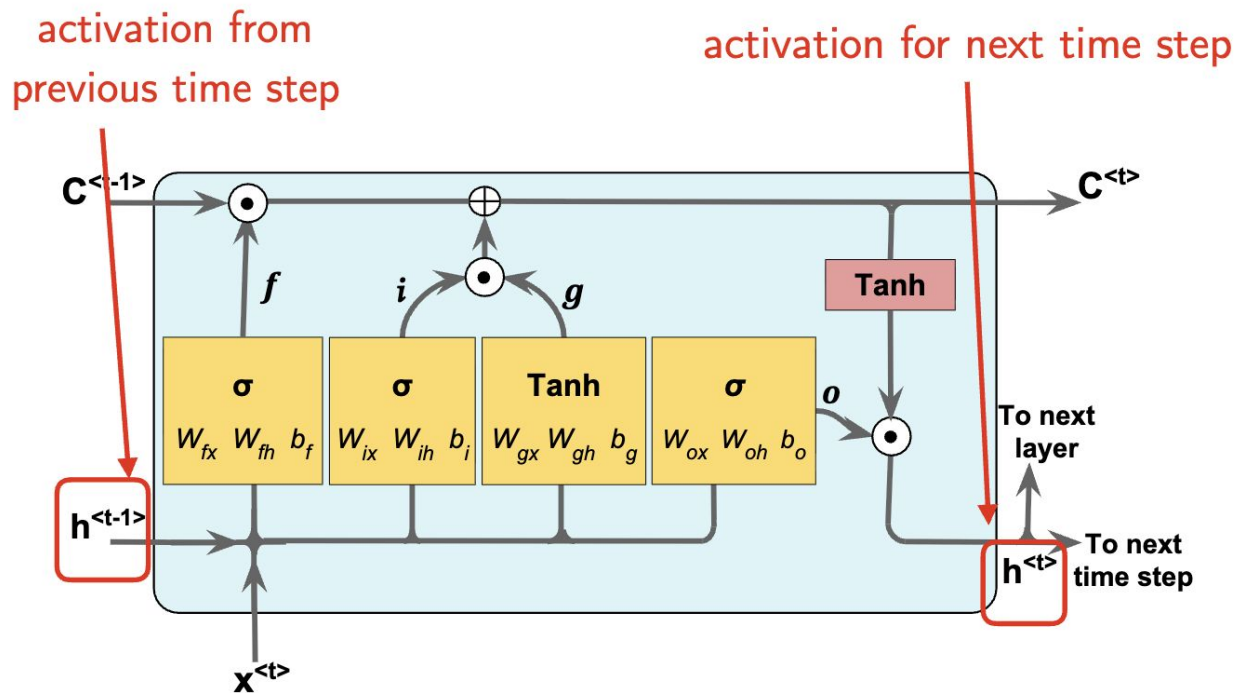
Zooming in: One LSTM Cell

Cell state at previous time step

Cell state at current time step



Zooming in: One LSTM Cell

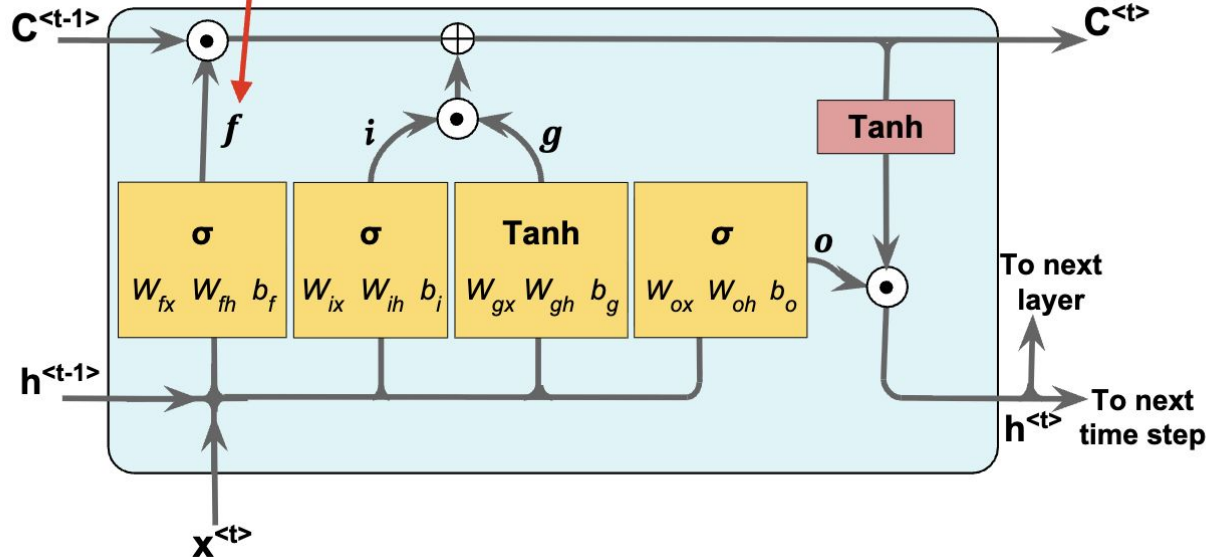


Forget Gate

"Forget Gate": controls which information is remembered, and which is forgotten;
can reset the cell state

Sigmoid output: $[0, 1]$

$$f_t = \sigma \left(\mathbf{W}_{fx} \mathbf{x}^{(t)} + \mathbf{W}_{fh} \mathbf{h}^{(t-1)} + \mathbf{b}_f \right)$$



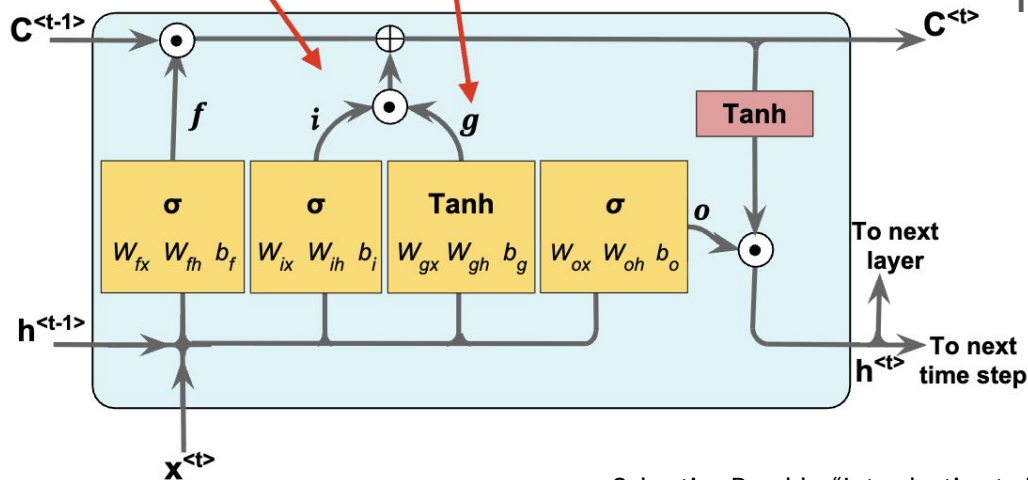
Input & Input Modulation

“Input Gate” $i_t = \sigma \left(\mathbf{W}_{ix} \mathbf{x}^{(t)} + \mathbf{W}_{ih} \mathbf{h}^{(t-1)} + \mathbf{b}_i \right)$ Sigmoid output: $[0, 1]$

“Input Modulation Gate”

$$\mathbf{g}_t = \tanh \left(\mathbf{W}_{gx} \mathbf{x}^{(t)} + \mathbf{W}_{gh} \mathbf{h}^{(t-1)} + \mathbf{b}_g \right)$$

Tanh output: $[-1, 1]$

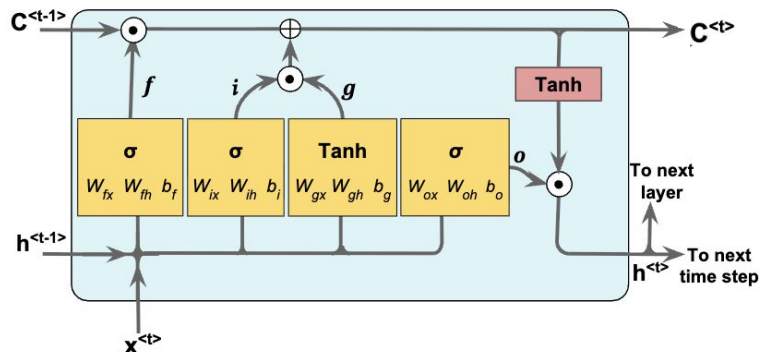


Summary: Updating the Cell State

Brief summary of the gates so far ...

$$C^{<t>} = \left(C^{<t-1>} \odot f_t \right) \oplus \underbrace{(i_t \odot g_t)}_{\text{How much of the new input we're taking}}$$

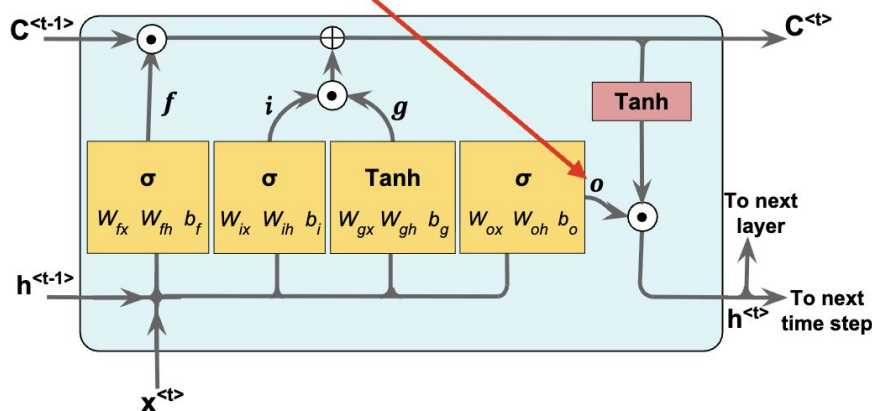
Forget Gate Input Gate Input Modulation Gate



Updating hidden units: Output gate

Output gate for updating the values of hidden units:

$$\mathbf{o}_t = \sigma \left(\mathbf{W}_{ox} \mathbf{x}^{(t)} + \mathbf{W}_{oh} \mathbf{h}^{(t-1)} + \mathbf{b}_o \right)$$



$$\mathbf{h}^{(t)} = \mathbf{o}_t \odot \tanh(\mathbf{C}^{(t)})$$

How do we choose tanh vs. sigmoid? (Briefly)

We choose **tanh** when we want **zero-centering**. This allows us to encode both negative and positive through a tanh output—this is a natural way to represent *absence* of a feature.

Wouldn't a bias term fix this? This would force many inputs into the flattened-out regions of the curve—we want to be able to be consistent about which inputs give us signal.

This is especially important for accumulators (something whose update reads its own value) that involve the cell state: h update and cell state update.

Why not a translation of a sigmoid?
(Great question from last week).

In fact....

$$\textbf{Sigmoid: } \sigma(x) = \frac{1}{1+e^{-x}} \qquad \sigma(x) = \frac{1+\tanh(\frac{x}{2})}{2}$$

$$\textbf{Tanh: } \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \qquad \tanh(x) = 2\sigma(2x) - 1$$

Let's try it out! A2 Part 2

The LSTM cell update is given below, where x_t is the input at time t , h_{t-1} is the previous hidden state, c_t is the memory cell, and $\odot$ is elementwise multiplication.

$$\square_1 = \sigma(W_1[h_{t-1}, x_t] + b_1)$$

$$\square_2 = \sigma(W_2[h_{t-1}, x_t] + b_2)$$

$$\square_3 = \sigma(W_3[h_{t-1}, x_t] + b_3)$$

$$\square_4 = \tanh(W_4[h_{t-1}, x_t] + b_4)$$

$$c_t = \square_3 \odot c_{t-1} + \square_1 \odot \square_4$$

$$h_t = \square_2 \odot \tanh(c_t)$$

(a) Each of $\square_1$ – $\square_4$ is one of the four LSTM gates. Identify each.

$\square_1$: _____

$\square_2$: _____

$\square_3$: _____

$\square_4$: _____

Options:

input gate (i)

forget gate (f)

output gate (o)

input modulation gate (g)

Let's try it out! A2 Part 2

The LSTM cell update is given below, where x_t is the input at time t , h_{t-1} is the previous hidden state, c_t is the memory cell, and $\odot$ is elementwise multiplication.

$$\square_1 = \sigma(W_1[h_{t-1}, x_t] + b_1)$$

$$\square_2 = \sigma(W_2[h_{t-1}, x_t] + b_2)$$

$$\square_3 = \sigma(W_3[h_{t-1}, x_t] + b_3)$$

$$\square_4 = \tanh(W_4[h_{t-1}, x_t] + b_4)$$

$$c_t = \square_3 \odot c_{t-1} + \square_1 \odot \square_4$$

$$h_t = \square_2 \odot \tanh(c_t)$$

(a) Each of $\square_1$ – $\square_4$ is one of the four LSTM gates. Identify each.

$\square_1$: input gate (i)

$\square_2$: output gate (o)

$\square_3$: forget gate (f)

$\square_4$: input modulation gate (g)

Let's try it out! A2 Part 2

(b) Fill in each blank. Use each option *at most once*; not every option is used.

Options: i (input gate) $\cdot$ f (forget gate) $\cdot$ o (output gate) $\cdot$ g (input modulation gate) $\cdot$ r (reset gate) $\cdot$ $(0, 1)$ $\cdot$ $(-1, 1)$ $\cdot$ c_{t-1} $\cdot$ h_{t-1} $\cdot$ $\tanh$ $\cdot$ σ $\cdot$ discarded $\cdot$ retained $\cdot$ overwritten

The cell is updated as

$$c_t = f \odot \text{_____} + \text{_____} \odot g,$$

so c_t has two additive contributions: the old contents, which are _____ when the corresponding value of f is near 0; and a new candidate value, which is admitted only when the corresponding gate value of _____ is near 1.

The hidden state is

$$h_t = o \odot \text{_____}(c_t).$$

Setting o to 0 means that c_t is _____ inside the cell but is not exposed to the next layer or to timestep $t + 1$'s input.